

AGENTE AUTÔNOMO PARA ANÁLISE EXPLORATÓRIA DE DADOS (EDA)

Relatório Técnico

Adriana Rocha Castro de Paula

Projeto do grupo Synapse 7 - formação em Agentes Autônomos

Versão revisada: 28 de julho de 2026

Aplicação pública

<https://agente-eda-app-adryrocha25.streamlit.app/>

Código-fonte

<https://github.com/AdryRocha/agente-eda-streamlit>

Tecnologias centrais

Python, Streamlit, LangChain, Gemini, Ollama, Pandas, Matplotlib e Seaborn

Nota de versão

Este documento substitui o relatório original de setembro de 2025 como referência técnica da versão atual. O código-fonte no repositório permanece a fonte definitiva para detalhes de implementação.

Resumo executivo

O projeto implementa um agente autônomo para Análise Exploratória de Dados (EDA) que permite carregar arquivos CSV e conversar com os dados em linguagem natural. O agente utiliza o padrão ReAct para selecionar ferramentas de análise, gerar estatísticas e visualizações e consolidar conclusões usando memória conversacional.

A solução foi atualizada para execução pública no Streamlit Community Cloud com Google Gemini, mantendo uma alternativa local com Ollama. A versão atual emprega o modelo gemini-3.6-flash, credenciais protegidas por Secrets, mensagens de erro adequadas ao usuário e controles de robustez para evitar loops ou exposição de detalhes internos.

Os testes mais recentes confirmaram o funcionamento do fluxo completo: upload do CSV, leitura do DataFrame, seleção autônoma de ferramentas, geração de histograma e mapa de correlação, contagem de classes e síntese das descobertas anteriores.

1. Objetivo e escopo

O objetivo do agente é democratizar a análise exploratória, permitindo que pessoas sem domínio de programação obtenham uma visão inicial de um conjunto de dados tabular. A aplicação foi desenhada como ferramenta de apoio à análise, e não como substituta da validação técnica por um profissional de dados.

- Receber arquivos CSV enviados pelo usuário.
- Interpretar perguntas em português por meio de um modelo de linguagem.
- Selecionar e executar ferramentas de EDA de forma autônoma.
- Exibir respostas textuais e gráficos diretamente na interface.
- Manter contexto recente para produzir conclusões consolidadas.
- Oferecer execução em nuvem e alternativa local voltada à privacidade.

2. Arquitetura da solução

Arquitetura atual da solução

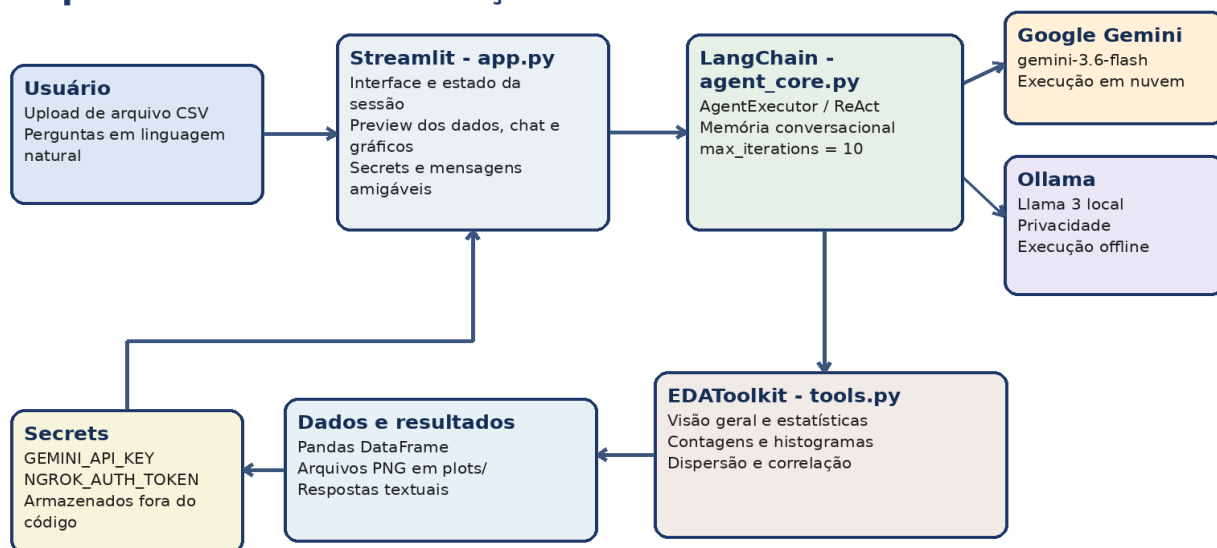


Figura 1 - Fluxo atual da aplicação, do upload do CSV às ferramentas e aos modelos de linguagem.

2.1 Camada de interface - app.py

- Constrói a interface web com Streamlit e gerencia o estado da sessão.
- Lê o CSV com Pandas e exibe uma prévia dos dados.
- Obtém a chave do Gemini a partir dos Secrets do Streamlit na implantação em nuvem.
- Mantém um modo local no qual o usuário pode optar por Ollama ou informar sua própria chave do Gemini.
- Renderiza respostas e imagens produzidas pelas ferramentas.
- Exibe mensagens amigáveis e envia o traceback completo apenas aos logs administrativos.

2.2 Núcleo de orquestração - agent_core.py

- Inicializa o Google Gemini ou o Ollama conforme o provedor selecionado.
- Usa o modelo gemini-3.6-flash na versão implantada em nuvem.
- Cria um agente ReAct com AgentExecutor e ferramentas especializadas.
- Mantém as cinco interações mais recentes com ConversationBufferWindowMemory.
- Define max_iterations=10 e handle_parsing_errors=True para reduzir loops e falhas de parsing.

2.3 Ferramentas de análise - tools.py

- Visão geral: dimensões, tipos, valores nulos e primeiras linhas.
- Estatísticas descritivas para variáveis numéricas.
- Contagem de valores de uma coluna.
- Histograma de uma variável numérica.
- Mapa de calor da matriz de correlação.
- Gráfico de dispersão entre duas colunas.
- Sanitização de nomes de colunas para remover espaços, aspas e quebras de linha enviados pelo modelo.

3. Tecnologias e decisões de projeto

Tecnologia	Papel	Justificativa
Streamlit	Interface e deploy	Permite criar aplicações de dados interativas e publicar rapidamente.
LangChain	Orquestração do agente	Integra LLM, prompt ReAct, memória e ferramentas.
Gemini	LLM em nuvem	Facilita a demonstração pública e o uso via API.
Ollama / Llama 3	LLM local	Oferece alternativa offline e maior controle de privacidade.
Pandas	Manipulação de dados	Converte o CSV em DataFrame e suporta as análises.
Matplotlib / Seaborn	Visualizações	Produzem histogramas, dispersão e mapas de correlação.
Streamlit Secrets / Colab Secrets	Credenciais	Evita chaves e tokens no código-fonte e no histórico do Git.

4. Segurança, privacidade e credenciais

A versão atual remove credenciais do código-fonte e separa a configuração por ambiente:

- Streamlit Community Cloud: GEMINI_API_KEY armazenada em Secrets no formato TOML.
- Google Colab: GEMINI_API_KEY e NGROK_AUTH_TOKEN cadastrados no painel Secrets.
- Execução local: uso de .streamlit/secrets.toml ou entrada manual da chave; o arquivo de Secrets deve permanecer no .gitignore.
- Ollama: não requer chave e permite processamento local, sendo a opção recomendada para dados confidenciais.

A demonstração pública não deve ser utilizada com informações pessoais, sigilosas ou reguladas. Arquivos enviados são processados no ambiente da sessão e podem envolver uma API externa quando o Gemini é usado.

```
GEMINI_API_KEY = "SUA_CHAVE"
```

```
# Nunca publicar .streamlit/secrets.toml, .env ou tokens em notebooks.
```

5. Atualizações realizadas desde o relatório original

Modelo Gemini: Migração do identificador experimental indisponível para gemini-3.6-flash, validado pela listagem de modelos disponível para a chave do projeto.

Secrets: Chave do Gemini movida para Streamlit Secrets; notebook revisado para usar Colab Secrets.

Erros: Tracebacks deixaram de ser exibidos ao usuário e passaram a ficar apenas nos logs.

Limite da API: Tratamento amigável para situações de cota temporária do plano gratuito.

Robustez do agente: Inclusão de limite de iterações e tratamento de erros de parsing.

Entradas das ferramentas: Limpeza de espaços, aspas e quebras de linha nos nomes de colunas.

Colab: Notebook refeito para clonar a versão atual do GitHub, evitando duplicação e desatualização do código.

6. Metodologia de teste

O teste funcional mais recente utilizou o conjunto público de transações de cartão de crédito com 284.807 registros e 31 colunas. A sequência abaixo foi executada em uma janela anônima, utilizando a aplicação publicada:

1. Carregar o arquivo CSV e iniciar o agente.
2. Solicitar uma descrição geral dos dados.
3. Solicitar um histograma da coluna Amount.
4. Solicitar análise de correlação entre as variáveis.
5. Solicitar a contagem de valores da coluna Class.
6. Pedir uma síntese das principais conclusões obtidas.

7. Evidências de execução

🤖 Agente Autônomo para Análise Exploratória de Dados (EDA) ⇄

Faça upload de um arquivo CSV e converse com seus dados em linguagem natural.



✓ Agente inicializado com Google Gemini (API, Grátis com Chave). Estou pronto para analisar!



Faça uma descrição geral dos dados.



Aqui está uma descrição geral do conjunto de dados:

1. Dimensões e Estrutura

- Total de Registros (Linhas): 284.807
- Total de Variáveis (Colunas): 31

Figura 2 - Inicialização com Gemini e resposta de visão geral do dataset.

Share ↗ ↻ ↺ ↻ ↺ ↻ ↺



Mostre-me a distribuição da coluna 'Amount' em um histograma.



O histograma da coluna `Amount` foi gerado com sucesso e salvo em `plots/histogram_Amount.png`.

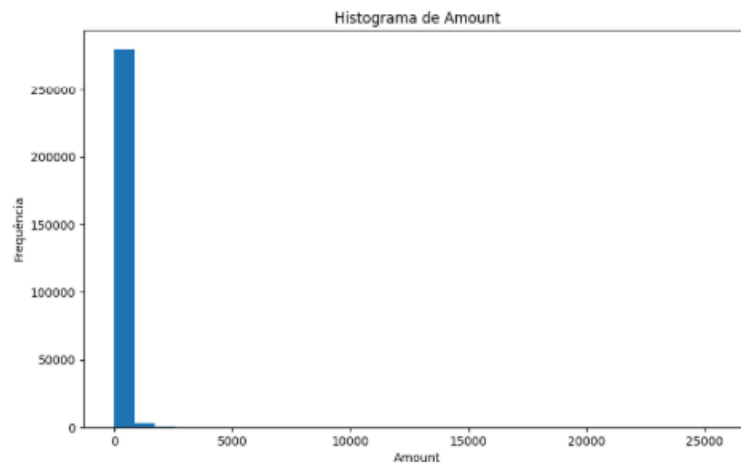


Figura 3 - Geração e exibição do histograma da coluna Amount.



Qual a contagem de valores para a coluna 'Class'?



A contagem de valores para a coluna `Class` é:

- `0` (Transações Legítimas): 284.315 (aproximadamente 99,83%)
- `1` (Transações Fraudulentas): 492 (aproximadamente 0,17%)

Total: 284.807 transações.



Com base nas análises que você fez, quais são suas principais conclusões sobre este dataset?



Com base nas análises realizadas até o momento, aqui estão as **principais conclusões e aprendizados** sobre este conjunto de dados de detecção de fraudes em cartão de crédito:

1. Qualidade e Integridade dos Dados

- **Dados Limpos e Completos:** O conjunto de dados possui 284.807 registros sem nenhum valor nulo ou ausente em suas 31 colunas, dispensando etapas de imputação de dados.

Figura 5 - Contagem da classe e início da síntese baseada no histórico.

2. Extremo Desbalanceamento de Classe (O Maior Desafio) ⇄

- **Proporção:** Apenas 0,17% das transações são fraudulentas (492 de 284.807).
- **Impacto na Modelagem:**
 - A métrica de **Acurácia** é enganosa (um modelo ingênuo que preveja sempre "legítimo" atingiria 99,83% de acurácia sem detectar nenhuma fraude).
 - Para avaliar modelos de forma eficaz, devem ser utilizadas métricas como **Precision, Recall, F1-Score, ROC-AUC e PR-AUC (Precision-Recall Area Under Curve)**.
 - É essencial aplicar técnicas para lidar com o desbalanceamento, como **Undersampling, Oversampling (SMOTE)** ou ajuste de **Class Weights** nos algoritmos.

3. Estrutura de Variáveis e Privacidade

- **PCA (V1 a V28):** As variáveis numéricas V1 a V28 já são componentes principais gerados por PCA. Isso garante a **ausência de multicolinearidade** (correlação zero entre elas), mas reduz a interpretabilidade direta dos recursos de negócio.
- **Preservação de Dados Brutos:** Apenas Amount (valor da transação) e Time (tempo decorrido) mantiveram sua forma original.

Figura 6 - Conclusões e recomendações apresentadas pelo agente.

8. Resultados observados

- O agente identificou corretamente as dimensões e os tipos de dados.
- A ausência de valores nulos foi reportada de forma coerente com o DataFrame.
- A classe alvo foi contada corretamente: 284.315 transações legítimas e 492 fraudulentas.
- O histograma e o mapa de calor foram produzidos e renderizados na interface.
- A memória permitiu consolidar descobertas das interações anteriores.
- O sistema continuou funcionando após uma resposta 429 temporária da API, com espera e nova tentativa.

9. Limitações e pontos de atenção

- O plano gratuito da API possui limites de frequência que podem variar por modelo, projeto e conta.
- Uma pergunta do agente pode consumir mais de uma chamada ao LLM: decisão da ferramenta e redação da resposta final.

- Arquivos muito grandes aumentam tempo de upload, uso de memória e duração das análises.
- O LLM pode acrescentar conhecimento geral não calculado pelas ferramentas. As conclusões devem ser confrontadas com os dados produzidos pelo código.
- Correlação não implica causalidade, e uma análise exploratória não substitui modelagem ou validação estatística completa.
- Dados confidenciais devem ser processados localmente ou em ambiente com governança apropriada.

10. Reprodutibilidade

A versão definitiva do notebook Colab não replica os arquivos Python dentro de células. Em vez disso, clona o repositório público e instala suas dependências, garantindo que a execução use a mesma versão do código disponível no GitHub.

```
git clone --depth 1 https://github.com/AdryRocha/agente-eda-streamlit.git
cd agente-eda-streamlit
pip install -r requirements.txt
streamlit run app.py
```

No Colab, a chave do Gemini e o token do ngrok são lidos do painel Secrets. Um arquivo secrets.toml temporário é criado somente na máquina virtual da sessão e não é enviado ao GitHub.

11. Conclusão

A versão atual atende ao objetivo proposto: disponibilizar um agente autônomo capaz de analisar arquivos CSV por linguagem natural, escolher ferramentas adequadas, gerar visualizações e produzir uma síntese contextual. O projeto demonstra integração de IA Generativa, agentes, engenharia de prompts, análise de dados, deploy e segurança de credenciais.

As melhorias implementadas após o relatório original tornaram a solução mais atual, reproduzível e adequada para apresentação como portfólio técnico. Os próximos avanços recomendados são observabilidade estruturada, validação automática das conclusões, amostragem para arquivos grandes e testes automatizados.

Referências e links do projeto

Aplicação: agente-eda-app-adryrocha25.streamlit.app

Repositório: github.com/AdryRocha/agente-eda-streamlit

Autora: Adriana Rocha Castro de Paula